

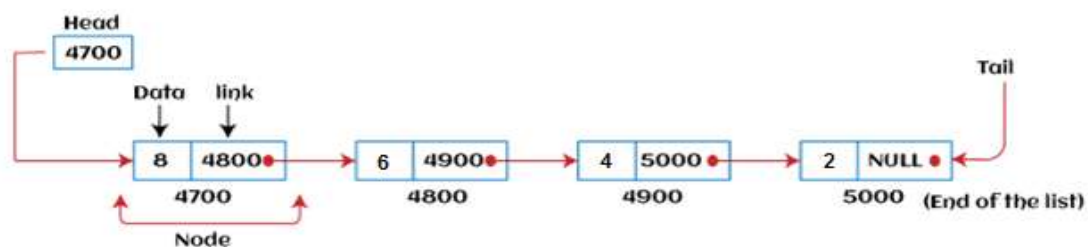
Chapter 3

Linked list

The linked list is a linear data structure that includes a series of connected nodes. A linked list can be defined as the nodes that are randomly stored in the memory. A node in the linked list contains two parts, i.e., the first is the data part and the second is the address part. The last node of the list contains a pointer to the null. After the array, the linked list is the second most used data structure. In a linked list, every link contains a connection to another link.

Representation of a Linked list

A linked list can be represented as the connection of nodes in which each node points to the next node of the list. The representation of the linked list is shown below -



Till now, we have been using array data structure to organize the group of elements that are to be stored individually in the memory. However, Array has several advantages and disadvantages that must be known to decide the data structure that will be used throughout the program.

Now, the question arises why we should use linked list over array?

Why use linked list over array?

Linked list is a data structure that overcomes the limitations of arrays. Let's first see some of the limitations of arrays -

- The size of the array must be known in advance before using it in the program.
- Increasing the size of the array is a time taking process. It is almost impossible to expand the size of the array at run time.
- All the elements in the array need to be contiguously stored in the memory. Inserting an element in the array needs shifting of all its predecessors.

Linked list is useful because -

- It allocates the memory dynamically. All the nodes of the linked list are non-contiguously stored in the memory and linked together with the help of pointers.

- In linked list, size is no longer a problem since we do not need to define its size at the time of declaration. List grows as per the program's demand and limited to the available memory space.

How to declare a linked list?

It is simple to declare an array, as it is of single type, while the declaration of linked list is a bit more typical than array. Linked list contains two parts, and both are of different types, i.e., one is the simple variable, while another is the pointer variable. We can declare the linked list by using the user-defined data type **structure**.

The declaration of linked list is given as follows -

```
struct node
{
    int data;
    struct node *next;
}
```

In the above declaration, we have defined a structure named as **node** that contains two variables, one is **data** that is of integer type, and another one is **next** that is a pointer which contains the address of next node.

Now, let's move towards the types of linked list.

Types of Linked list

Linked list is classified into the following types -

- **Singly-linked list** - Singly linked list can be defined as the collection of an ordered set of elements. A node in the singly linked list consists of two parts: data part and link part. Data part of the node stores actual information that is to be represented by the node, while the link part of the node stores the address of its immediate successor.
- **Doubly linked list** - Doubly linked list is a complex type of linked list in which a node contains a pointer to the previous as well as the next node in the sequence. Therefore, in a doubly-linked list, a node consists of three parts: node data, pointer to the next node in sequence (next pointer), and pointer to the previous node (previous pointer).
- **Circular singly linked list** - In a circular singly linked list, the last node of the list contains a pointer to the first node of the list. We can have circular singly linked list as well as circular doubly linked list.

- **Circular doubly linked list** - Circular doubly linked list is a more complex type of data structure in which a node contains pointers to its previous node as well as the next node. Circular doubly linked list doesn't contain NULL in any of the nodes. The last node of the list contains the address of the first node of the list. The first node of the list also contains the address of the last node in its previous pointer.

Now, let's see the benefits and limitations of using the Linked list.

Advantages of Linked list

The advantages of using the Linked list are given as follows -

- **Dynamic data structure** - The size of the linked list may vary according to the requirements. Linked list does not have a fixed size.
- **Insertion and deletion** - Unlike arrays, insertion, and deletion in linked list is easier. Array elements are stored in the consecutive location, whereas the elements in the linked list are stored at a random location. To insert or delete an element in an array, we have to shift the elements for creating the space. Whereas, in linked list, instead of shifting, we just have to update the address of the pointer of the node.
- **Memory efficient** - The size of a linked list can grow or shrink according to the requirements, so memory consumption in linked list is efficient.
- **Implementation** - We can implement both stacks and queues using linked list.

Disadvantages of Linked list

The limitations of using the Linked list are given as follows -

- **Memory usage** - In linked list, node occupies more memory than array. Each node of the linked list occupies two types of variables, i.e., one is a simple variable, and another one is the pointer variable.
- **Traversal** - Traversal is not easy in the linked list. If we have to access an element in the linked list, we cannot access it randomly, while in case of array we can randomly access it by index. For example, if we want to access the 3rd node, then we need to traverse all the nodes before it. So, the time required to access a particular node is large.
- **Reverse traversing** - Backtracking or reverse traversing is difficult in a linked list. In a doubly-linked list, it is easier but requires more memory to store the back pointer.

Applications of Linked list

The applications of the Linked list are given as follows -

- With the help of a linked list, the polynomials can be represented as well as we can perform the operations on the polynomial.
- A linked list can be used to represent the sparse matrix.
- The various operations like student's details, employee's details, or product details can be implemented using the linked list as the linked list uses the structure data type that can hold different data types.
- Using linked list, we can implement stack, queue, tree, and other various data structures.
- The graph is a collection of edges and vertices, and the graph can be represented as an adjacency matrix and adjacency list. If we want to represent the graph as an adjacency matrix, then it can be implemented as an array. If we want to represent the graph as an adjacency list, then it can be implemented as a linked list.
- A linked list can be used to implement dynamic memory allocation. The dynamic memory allocation is the memory allocation done at the run-time.

Operations performed on the Linked list

The basic operations that are supported by a list are mentioned as follows -

- **Insertion** - This operation is performed to add an element to the list.
- **Deletion** - It is performed to delete an operation from the list.
- **Display** - It is performed to display the elements of the list.
- **Search** - It is performed to search an element from the list using the given key.

Complexity of Linked list

Now, let's see the time and space complexity of the linked list for the operations search, insert, and delete.

1. Time Complexity

Operations	Average case time complexity	Worst-case time complexity
Insertion	$O(1)$	$O(1)$
Deletion	$O(1)$	$O(1)$

Search	$O(n)$	$O(n)$
---------------	--------	--------

Where 'n' is the number of nodes in the given tree.

Types of Linked List

Before knowing about the types of a linked list, we should know what is **linked list**. So, to know about the linked list, click on the link given below:

Types of Linked list

The following are the types of linked list:

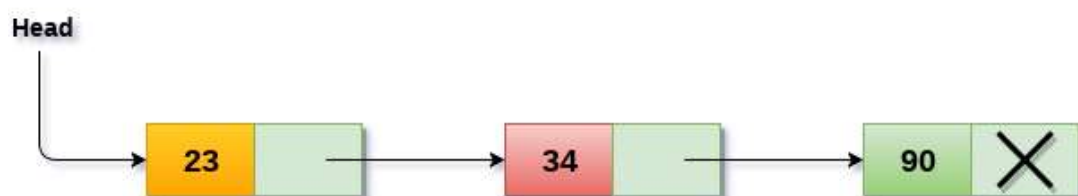
- Singly Linked list
- Doubly Linked list
- Circular Linked list
- Doubly Circular Linked list

1. Singly Linked List:

The singly-linked list can be defined as the collection of an ordered set of elements. The number of elements may vary according to the needs of the program. A node in the singly linked list consists of two parts: the data part and the linked part. The data part of the node stores actual information that is to be represented by the node while the link part of the node stores the address of its immediate successor.

A one-way chain or singly linked list can be traversed only in one direction. In other words, we can say that each node contains only the next pointer, therefore we cannot traverse the list in the reverse direction.

Consider an example where the marks obtained by the student in three subjects are stored in a linked list as shown in the figure.



In the above figure, the arrow represents the links. The data part of every node contains the marks obtained by the student in the different subject. The last node in the list is identified by the null pointer which is present in the address part of the last node. We can have as many elements we require, in the data part of the list.

Complexity

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Singly Linked List	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$

Operations on Singly Linked List

There are various operations which can be performed on singly linked list. A list of all such operations is given below.

1. Node Creation

```
struct node
{
    int data;
    struct node *next;
};
struct node *head, *ptr;
ptr = (struct node *)malloc(sizeof(struct node *));
```

2. Insertion

The insertion into a singly linked list can be performed at different positions. Based on the position of the new node being inserted, the insertion is categorized into the following categories.

SN	Operation	Description
1	Insertion at beginning	It involves inserting any element at the front of the list. We just need to a few link adjustments to make the new node as the head of the list.

2	Insertion at end of the list	It involves insertion at the last of the linked list. The new node can be inserted as the only node in the list or it can be inserted as the last one. Different logics are implemented in each scenario.
3	Insertion after specified node	It involves insertion after the specified node of the linked list. We need to skip the desired number of nodes in order to reach the node after which the new node will be inserted. .

Deletion and Traversing

The Deletion of a node from a singly linked list can be performed at different positions. Based on the position of the node being deleted, the operation is categorized into the following categories.

SN	Operation	Description
1	Deletion at beginning	It involves deletion of a node from the beginning of the list. This is the simplest operation among all. It just need a few adjustments in the node pointers.
2	Deletion at the end of the list	It involves deleting the last node of the list. The list can either be empty or full. Different logic is implemented for the different scenarios.
3	Deletion after specified node	It involves deleting the node after the specified node in the list. we need to skip the desired number of nodes to reach the node after which the node will be deleted. This requires traversing through the list.
4	Traversing	In traversing, we simply visit each node of the list at least once in order to perform some specific operation on it, for example, printing data part of each node present in the list.
5	Searching	In searching, we match each element of the list with the given element. If the element is found on any of the location, then location of that element is returned otherwise null is returned. .

1. Insertion:

Case 1: Insertion at Beginning:

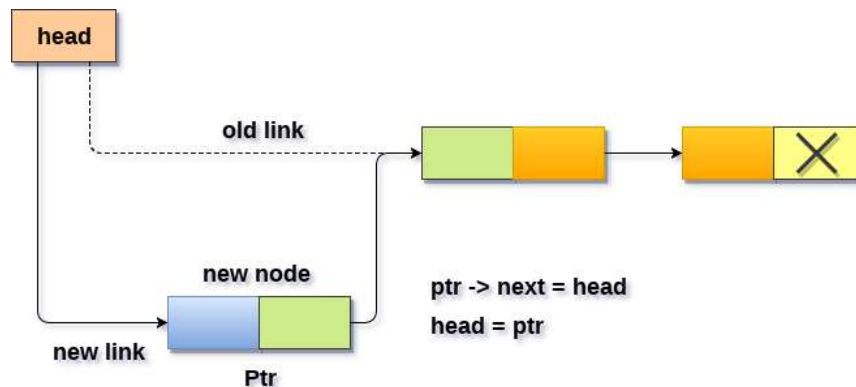
Insertion in singly linked list at beginning

Inserting a new element into a singly linked list at beginning is quite simple. We just need to make a few adjustments in the node links. There are the following steps which need to be followed in order to insert a new node in the list at beginning.

- Allocate the space for the new node and store data into the data part of the node. This will be done by the following statements.
 1. `ptr = (struct node *) malloc(sizeof(struct node *));`
 2. `ptr → data = item`
- Make the link part of the new node pointing to the existing first node of the list. This will be done by using the following statement.
 1. `ptr->next = head;`
- At the last, we need to make the new node as the first node of the list this will be done by using the following statement.
 1. `head = ptr;`

Algorithm

- **Step 1:** IF PTR = NULL Write OVERFLOW
 - Go to Step 7
 - [END OF IF]
- **Step 2:** SET NEW_NODE = PTR
- **Step 3:** SET PTR = PTR → NEXT
- **Step 4:** SET NEW_NODE → DATA = VAL
- **Step 5:** SET NEW_NODE → NEXT = HEAD
- **Step 6:** SET HEAD = NEW_NODE
- **Step 7:** EXIT



Case 2: Insertion in singly linked list at the end

In order to insert a node at the last, there are two following scenarios which need to be mentioned.

1. The node is being added to an empty list
2. The node is being added to the end of the linked list

in the first case,

- The condition (`head == NULL`) gets satisfied. Hence, we just need to allocate the space for the node by using `malloc` statement in C. Data and the link part of the node are set up by using the following statements.

```
ptr->data = item;
ptr -> next = NULL;
```

- Since, **ptr** is the only node that will be inserted in the list hence, we need to make this node pointed by the head pointer of the list. This will be done by using the following Statements.
- `Head = ptr`

In the second case,

- The condition **Head = NULL** would fail, since Head is not null. Now, we need to declare a temporary pointer `temp` in order to traverse through the list. **temp** is made to point the first node of the list.

```
Temp = head
```

Then, traverse through the entire linked list using the statements:

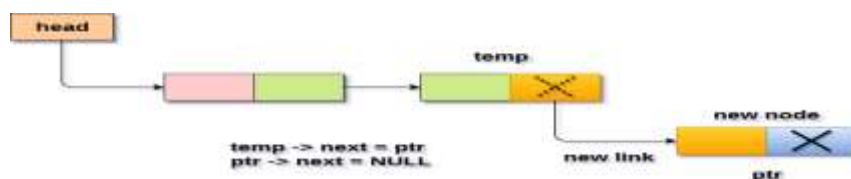
```
while (temp->next != NULL)
temp = temp -> next;
```

- At the end of the loop, the temp will be pointing to the last node of the list. Now, allocate the space for the new node, and assign the item to its data part. Since, the new node is going to be the last node of the list hence, the next part of this node needs to be pointing to the **null**. We need to make the next part of the temp node (which is currently the last node of the list) point to the new node (ptr) .

```
temp = head;
while (temp -> next != NULL)
{
    temp = temp -> next;
}
temp->next = ptr;
ptr->next = NULL;
```

Algorithm

- **Step 1:** IF PTR = NULL Write OVERFLOW
Go to Step 1
[END OF IF]
- **Step 2:** SET NEW_NODE = PTR
- **Step 3:** SET PTR = PTR - > NEXT
- **Step 4:** SET NEW_NODE - > DATA = VAL
- **Step 5:** SET NEW_NODE - > NEXT = NULL
- **Step 6:** SET PTR = HEAD
- **Step 7:** Repeat Step 8 while PTR - > NEXT != NULL
- **Step 8:** SET PTR = PTR - > NEXT
[END OF LOOP]
- **Step 9:** SET PTR - > NEXT = NEW_NODE
- **Step 10:** EXIT



Inserting node at the last into a non-empty list

Insertion in singly linked list after specified Node

- In order to insert an element after the specified number of nodes into the linked list, we need to skip the desired number of elements in the list to move the pointer at the position after which the node will be inserted. This will be done by using the following statements.

```
emp=head;
for(i=0;i<loc;i++)
{
    temp = temp->next;
    if(temp == NULL)
    {
        return;
    }
}
```

- Allocate the space for the new node and add the item to the data part of it. This will be done by using the following statements.

```
ptr = (struct node *) malloc (sizeof(struct node));
ptr->data = item;
```

- Now, we just need to make a few more link adjustments and our node at will be inserted at the specified position. Since, at the end of the loop, the loop pointer temp would be pointing to the node after which the new node will be inserted. Therefore, the next part of the new node ptr must contain the address of the next part of the temp (since, ptr will be in between temp and the next of the temp). This will be done by using the following statements.

ptr→ next = temp → next

now, we just need to make the next part of the temp, point to the new node ptr. This will insert the new node ptr, at the specified position.

temp ->next = ptr;

Algorithm

- **STEP 1:** IF PTR = NULL

WRITE OVERFLOW

GOTO STEP 12

END OF IF

- **STEP 2:** SET NEW_NODE = PTR
- **STEP 3:** NEW_NODE → DATA = VAL
- **STEP 4:** SET TEMP = HEAD
- **STEP 5:** SET I = 0
- **STEP 6:** REPEAT STEP 5 AND 6 UNTIL I<loc< li=""></loc>- **STEP 7:** TEMP = TEMP → NEXT
- **STEP 8:** IF TEMP = NULL

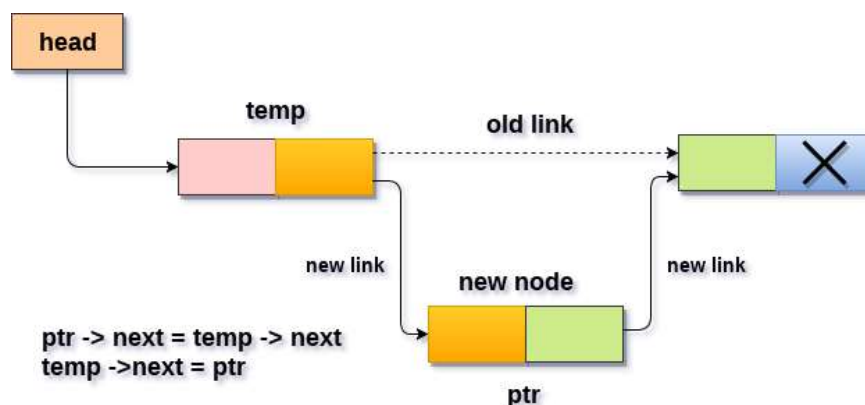
WRITE "DESIRED NODE NOT PRESENT"

GOTO STEP 12

END OF IF

END OF LOOP

- **STEP 9:** PTR → NEXT = TEMP → NEXT
- **STEP 10:** TEMP → NEXT = PTR
- **STEP 11:** SET PTR = NEW_NODE
- **STEP 12:** EXIT



Deletion in singly linked list at beginning

Deleting a node from the beginning of the list is the simplest operation of all. It just need a few adjustments in the node pointers. Since the first node of the list is to be deleted, therefore, we just need to make the head, point to the next of the head. This will be done by using the following statements.

```
ptr = head;  
head = ptr->next;
```

Now, free the pointer ptr which was pointing to the head node of the list. This will be done by using the following statement.

```
free(ptr)
```

Algorithm

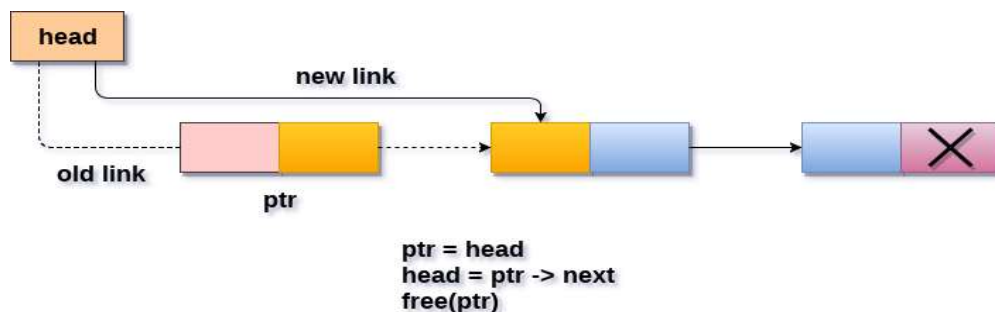
- **Step 1:** IF HEAD = NULL

Write UNDERFLOW

Go to Step 5

[END OF IF]

- **Step 2:** SET PTR = HEAD
- **Step 3:** SET HEAD = HEAD -> NEXT
- **Step 4:** FREE PTR
- **Step 5:** EXIT



Deleting a node from the beginning

Deletion in singly linked list at the end

There are two scenarios in which, a node is deleted from the end of the linked list.

1. There is only one node in the list and that needs to be deleted.
2. There is more than one node in the list and the last node of the list will be deleted.

In the first scenario,

the condition $\text{head} \rightarrow \text{next} = \text{NULL}$ will survive and therefore, the only node head of the list will be assigned to null. This will be done by using the following statements.

```
ptr = head
head = NULL
free(ptr)
```

In the second scenario,

The condition $\text{head} \rightarrow \text{next} = \text{NULL}$ would fail and therefore, we have to traverse the node in order to reach the last node of the list.

For this purpose, just declare a temporary pointer temp and assign it to head of the list. We also need to keep track of the second last node of the list. For this purpose, two pointers ptr and ptr1 will be used where ptr will point to the last node and ptr1 will point to the second last node of the list.

this all will be done by using the following statements.

```
ptr = head;
while(ptr->next != NULL)
{
    ptr1 = ptr;
    ptr = ptr ->next;
}
```

Now, we just need to make the pointer ptr1 point to the NULL and the last node of the list that is pointed by ptr will become free. It will be done by using the following statements.

```
ptr1->next = NULL;
free(ptr);
```

Algorithm

- **Step 1:** IF HEAD = NULL

Write UNDERFLOW

Go to Step 8

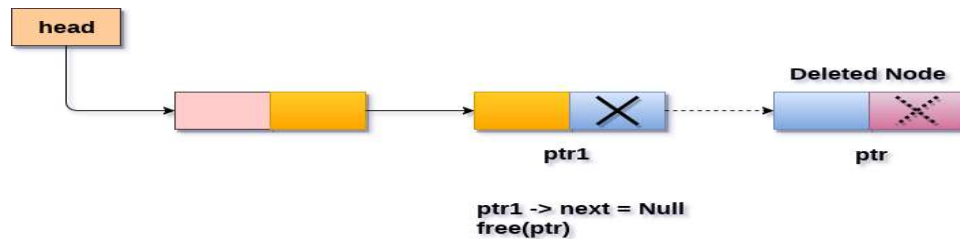
[END OF IF]

- **Step 2:** SET PTR = HEAD
- **Step 3:** Repeat Steps 4 and 5 while PTR -> NEXT!= NULL
- **Step 4:** SET PREPTR = PTR

- **Step 5:** SET PTR = PTR -> NEXT

[END OF LOOP]

- **Step 6:** SET PREPTR -> NEXT = NULL
- **Step 7:** FREE PTR
- **Step 8:** EXIT



Deleting a node from the last

Deletion in singly linked list after the specified node:

In order to delete the node, which is present after the specified node, we need to skip the desired number of nodes to reach the node after which the node will be deleted. We need to keep track of the two nodes. The one which is to be deleted the other one if the node which is present before that node. For this purpose, two pointers are used: ptr and ptr1.

Use the following statements to do so.

```
ptr=head;
for(i=0;i<loc;i++)
{
    ptr1 = ptr;
    ptr = ptr->next;

    if(ptr == NULL)
    {
        printf("\nThere are less than %d elements in the list..",loc);
        return;
    }
}
```

Now, our task is almost done, we just need to make a few pointer adjustments. Make the next of ptr1 (points to the specified node) point to the next of ptr (the node which is to be deleted).

This will be done by using the following statements.

```
ptr1 ->next = ptr ->next;  
free(ptr);
```

Algorithm

- **STEP 1:** IF HEAD = NULL

WRITE UNDERFLOW

GOTO STEP 10

END OF IF

- **STEP 2:** SET TEMP = HEAD
- **STEP 3:** SET I = 0
- **STEP 4:** REPEAT STEP 5 TO 8 UNTIL I < loc
- **STEP 5:** TEMP1 = TEMP
- **STEP 6:** TEMP = TEMP → NEXT
- **STEP 7:** IF TEMP = NULL

WRITE "DESIRED NODE NOT PRESENT"

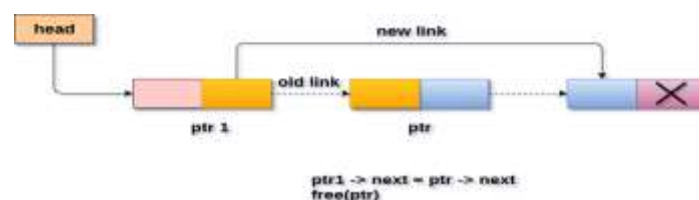
GOTO STEP 12

END OF IF

- **STEP 8:** I = I+1

END OF LOOP

- **STEP 9:** TEMP1 → NEXT = TEMP → NEXT
- **STEP 10:** FREE TEMP
- **STEP 11:** EXIT



Deletion a node from specified position

Traversing in singly linked list

Traversing is the most common operation that is performed in almost every scenario of singly linked list. Traversing means visiting each node of the list once in order to perform some operation on that. This will be done by using the following statements.

```
ptr = head;
while (ptr!=NULL)
{
    ptr = ptr -> next;
}
```

Algorithm

- **STEP 1:** SET PTR = HEAD
- **STEP 2:** IF PTR = NULL

 WRITE "EMPTY LIST"
 GOTO STEP 7
 END OF IF
- **STEP 4:** REPEAT STEP 5 AND 6 UNTIL PTR != NULL
- **STEP 5:** PRINT PTR → DATA
- **STEP 6:** PTR = PTR → NEXT

 [END OF LOOP]
- **STEP 7:** EXIT

Searching in singly linked list

Searching is performed in order to find the location of a particular element in the list. Searching any element in the list needs traversing through the list and make the comparison of every element of the list with the specified element. If the element is matched with any of the list element, then the location of the element is returned from the function.

Algorithm

- **Step 1:** SET PTR = HEAD
- **Step 2:** Set I = 0
- **STEP 3:** IF PTR = NULL

 WRITE "EMPTY LIST"
 GOTO STEP 8
 END OF IF

- **STEP 4:** REPEAT STEP 5 TO 7 UNTIL PTR != NULL
- **STEP 5:** if ptr → data = item

write i+1

End of IF

- **STEP 6:** I = I + 1
- **STEP 7:** PTR = PTR → NEXT

[END OF LOOP]

- **STEP 8:** EXIT

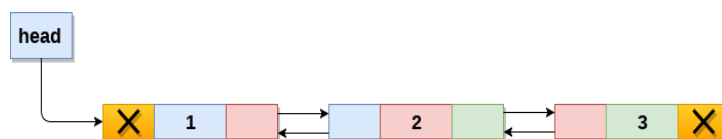
Doubly linked list

Doubly linked list is a complex type of linked list in which a node contains a pointer to the previous as well as the next node in the sequence. Therefore, in a doubly linked list, a node consists of three parts: node data, pointer to the next node in sequence (next pointer) , pointer to the previous node (previous pointer). A sample node in a doubly linked list is shown in the figure.



Node

A doubly linked list containing three nodes having numbers from 1 to 3 in their data part, is shown in the following image.



Doubly Linked List

In C, structure of a node in doubly linked list can be given as :

```
struct node
{
    struct node *prev;
```

```

        int data;
        struct node *next;
    }

```

The **prev** part of the first node and the **next** part of the last node will always contain null indicating end in each direction.

In a singly linked list, we could traverse only in one direction, because each node contains address of the next node and it doesn't have any record of its previous nodes. However, doubly linked list overcome this limitation of singly linked list. Due to the fact that, each node of the list contains the address of its previous node, we can find all the details about the previous node as well by using the previous address stored inside the previous part of each node.

Memory Representation of a doubly linked list

Memory Representation of a doubly linked list is shown in the following image. Generally, doubly linked list consumes more space for every node and therefore, causes more expansive basic operations such as insertion and deletion. However, we can easily manipulate the elements of the list since the list maintains pointers in both the directions (forward and backward).

In the following image, the first element of the list that is i.e. 13 stored at address 1. The head pointer points to the starting address 1. Since this is the first element being added to the list therefore the **prev** of the list **contains** null. The next node of the list resides at address 4 therefore the first node contains 4 in its next pointer.

We can traverse the list in this way until we find any node containing null or -1 in its next part.

Operations on doubly linked list

Node Creation

```

struct node
{
    struct node *prev;
    int data;
    struct node *next;
};
struct node *head;

```

All the remaining operations regarding doubly linked list are described in the following table.

SN	Operation	Description
----	-----------	-------------

1	<u>Insertion at beginning</u>	Adding the node into the linked list at beginning.
2	<u>Insertion at end</u>	Adding the node into the linked list to the end.
3	<u>Insertion after specified node</u>	Adding the node into the linked list after the specified node.
4	<u>Deletion at beginning</u>	Removing the node from beginning of the list
5	<u>Deletion at the end</u>	Removing the node from end of the list.
6	<u>Deletion of the node having given data</u>	Removing the node which is present just after the node containing the given data.
7	<u>Searching</u>	Comparing each node data with the item to be searched and return the location of the item in the list if the item found else return null.
8	<u>Traversing</u>	Visiting each node of the list at least once in order to perform some specific operation like searching, sorting, display, etc.

Insertion in doubly linked list at beginning

As in doubly linked list, each node of the list contains double pointers therefore we have to maintain more number of pointers in doubly linked list as compare to singly linked list.

There are two scenarios of inserting any element into doubly linked list. Either the list is empty or it contains at least one element. Perform the following steps to insert a node in doubly linked list at beginning.

- Allocate the space for the new node in the memory. This will be done by using the following statement.

```
ptr = (struct node *)malloc(sizeof(struct node));
```

- Check whether the list is empty or not. The list is empty if the condition `head == NULL` holds. In that case, the node will be inserted as the only node of the list and therefore the prev and the next pointer of the node will point to NULL and the head pointer will point to this node.

```
ptr->next = NULL;
```

```
ptr->prev=NULL;
ptr->data=item;
head=ptr;
```

- In the second scenario, the condition **head == NULL** become false and the node will be inserted in beginning. The next pointer of the node will point to the existing head pointer of the node. The prev pointer of the existing head will point to the new node being inserted.
- This will be done by using the following statements.

```
ptr->next = head;
head->prev=ptr;
```

Since, the node being inserted is the first node of the list and therefore it must contain NULL in its prev pointer. Hence assign null to its previous part and make the head point to this node.

```
ptr->prev =NULL
head = ptr
```

Algorithm :

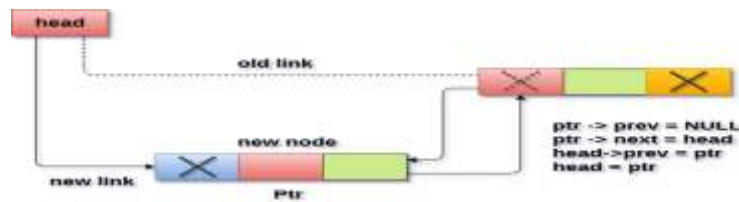
- **Step 1:** IF ptr = NULL

Write OVERFLOW

Go to Step 9

[END OF IF]

- **Step 2:** SET NEW_NODE = ptr
- **Step 3:** SET ptr = ptr -> NEXT
- **Step 4:** SET NEW_NODE -> DATA = VAL
- **Step 5:** SET NEW_NODE -> PREV = NULL
- **Step 6:** SET NEW_NODE -> NEXT = START
- **Step 7:** SET head -> PREV = NEW_NODE
- **Step 8:** SET head = NEW_NODE
- **Step 9:** EXIT



2. Insertion into doubly linked list at beginning

Insertion in doubly linked list at the end

In order to insert a node in doubly linked list at the end, we must make sure whether the list is empty or it contains any element. Use the following steps in order to insert the node in doubly linked list at the end.

- Allocate the memory for the new node. Make the pointer **ptr** point to the new node being inserted.

```
ptr = (struct node *) malloc(sizeof(struct node));
```

- Check whether the list is empty or not. The list is empty if the condition **head == NULL** holds. In that case, the node will be inserted as the only node of the list and therefore the prev and the next pointer of the node will point to NULL and the head pointer will point to this node.

```
ptr->next = NULL;
ptr->prev=NULL;
ptr->data=item;
head=ptr;
```

- In the second scenario, the condition **head == NULL** become false. The new node will be inserted as the last node of the list. For this purpose, we have to traverse the whole list in order to reach the last node of the list. Initialize the pointer **temp** to head and traverse the list by using this pointer.

```
Temp = head;
while (temp != NULL)
{
    temp = temp -> next;
}
```

the pointer temp point to the last node at the end of this while loop. Now, we just need to make a few pointer adjustments to insert the new node ptr to the list. First, make the next pointer of temp point to the new node being inserted i.e. ptr.

temp → next = ptr;

make the previous pointer of the node ptr point to the existing last node of the list i.e. temp.

ptr → prev = temp;

make the next pointer of the node ptr point to the null as it will be the new last node of the list.

ptr → next = NULL

Algorithm

- **Step 1:** IF PTR = NULL

Write OVERFLOW

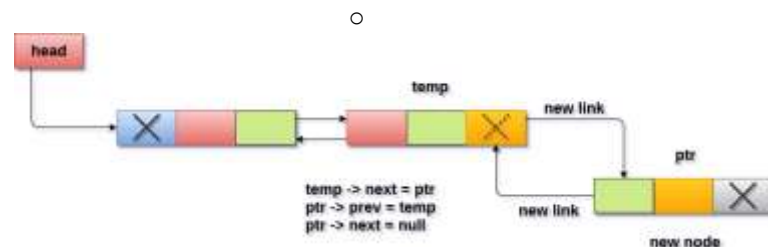
Go to Step 11

[END OF IF]

- **Step 2:** SET NEW_NODE = PTR
- **Step 3:** SET PTR = PTR → NEXT
- **Step 4:** SET NEW_NODE → DATA = VAL
- **Step 5:** SET NEW_NODE → NEXT = NULL
- **Step 6:** SET TEMP = START
- **Step 7:** Repeat Step 8 while TEMP → NEXT != NULL
- **Step 8:** SET TEMP = TEMP → NEXT

[END OF LOOP]

- **Step 9:** SET TEMP → NEXT = NEW_NODE
- **Step 10C:** SET NEW_NODE → PREV = TEMP
- **Step 11:** EXIT



Insertion into doubly linked list at the end

Insertion in doubly linked list after Specified node

In order to insert a node after the specified node in the list, we need to skip the required number of nodes in order to reach the mentioned node and then make the pointer adjustments as required.

Use the following steps for this purpose.

- Allocate the memory for the new node. Use the following statements for this.

```
ptr = (struct node *)malloc(sizeof(struct node));
```

- Traverse the list by using the pointer **temp** to skip the required number of nodes in order to reach the specified node.

```
temp=head;
for(i=0;i<loc;i++)
{
    temp = temp->next;
    if(temp == NULL) // the temp will be //null if the list doesn't last long //up
to mentioned location
    {
        return;
    }
}
```

- The temp would point to the specified node at the end of the **for** loop. The new node needs to be inserted after this node therefore we need to make a few pointer adjustments here. Make the next pointer of **ptr** point to the next node of temp.

```
ptr → next = temp → next;
```

make the **prev** of the new node ptr point to temp.

```
ptr → prev = temp;
```

make the **next** pointer of temp point to the new node ptr.

```
temp → next = ptr;
```

make the **previous** pointer of the next node of temp point to the new node.

```
temp → next → prev = ptr;
```

Algorithm

- **Step 1:** IF PTR = NULL

Write OVERFLOW

Go to Step 15

[END OF IF]

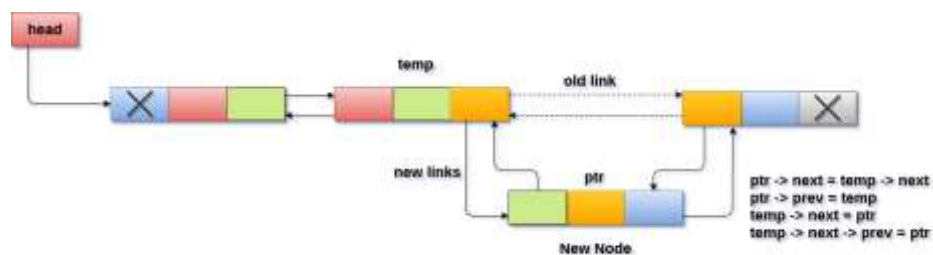
- **Step 2:** SET NEW_NODE = PTR
- **Step 3:** SET PTR = PTR -> NEXT
- **Step 4:** SET NEW_NODE -> DATA = VAL
- **Step 5:** SET TEMP = START
- **Step 6:** SET I = 0
- **Step 7:** REPEAT 8 to 10 until I <= "" li = "" >
- **Step 8:** SET TEMP = TEMP -> NEXT
- **STEP 9:** IF TEMP = NULL
- **STEP 10:** WRITE "LESS THAN DESIRED NO. OF ELEMENTS"

GOTO STEP 15

[END OF IF]

[END OF LOOP]

- **Step 11:** SET NEW_NODE -> NEXT = TEMP -> NEXT
- **Step 12:** SET NEW_NODE -> PREV = TEMP
- **Step 13 :** SET TEMP -> NEXT = NEW_NODE
- **Step 14:** SET TEMP -> NEXT -> PREV = NEW_NODE
- **Step 15:** EXIT



Insertion into doubly linked list after specified node

Deletion in doubly linked list at the beginning is the simplest operation. We just need to copy the head pointer to pointer ptr and shift the head pointer to its next.

now make the prev of this new head node point to NULL. This will be done by using the following statements.

Now free the pointer `ptr` by using the **free** function.

Algorithm

- ```
WRITE UNDERFLOW
GOTO STEP 6
```

- 
- Diagram illustrating the deletion of a node from a doubly linked list:
- The **HEAD** pointer is updated to point to the next node (labeled **new link**).
  - The **deleted node** is shown with a red segment (containing an 'X') and a blue segment.
  - The **old link** (dashed arrow) points from the **HEAD** to the **deleted node**.
  - The next node is shown with a red segment (containing an 'X') and a blue segment.
  - The **Head** pointer now points to the next node.
- Code snippet for deletion:
- ```
ptr = head
head = head -> next
head -> prev = NULL
free(ptr)
```

Deletion in doubly linked list from beginning

Deletion in doubly linked list at the end

Deletion of the last node in a doubly linked list needs traversing the list in order to reach the last node of the list and then make pointer adjustments at that position.

In order to delete the last node of the list, we need to follow the following steps.

- If the list is already empty then the condition `head == NULL` will become true and therefore the operation can not be carried on.
- If there is only one node in the list then the condition `head → next == NULL` become true. In this case, we just need to assign the head of the list to NULL and free head in order to completely delete the list.
- Otherwise, just traverse the list to reach the last node of the list. This will be done by using the following statements.

```
1. ptr = head;
2.   if(ptr->next != NULL)
3.   {
4.       ptr = ptr -> next;
5.   }
```

- The ptr would point to the last node of the list at the end of the for loop. Just make the next pointer of the previous node of **ptr** to **NULL**.

```
1. ptr → prev → next = NULL
```

free the pointer as this the node which is to be deleted.

```
1. free(ptr)
```

- **Step 1:** IF HEAD = NULL

Write UNDERFLOW

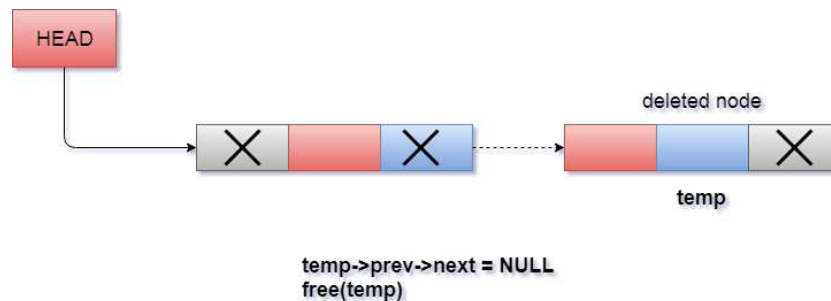
Go to Step 7

[END OF IF]

- **Step 2:** SET TEMP = HEAD
- **Step 3:** REPEAT STEP 4 WHILE TEMP->NEXT != NULL
- **Step 4:** SET TEMP = TEMP->NEXT

[END OF LOOP]

- **Step 5:** SET TEMP -> PREV -> NEXT = NULL
- **Step 6:** FREE TEMP
- **Step 7:** EXIT



Deletion in doubly linked list at the end

Deletion in doubly linked list after the specified node

In order to delete the node after the specified data, we need to perform the following steps.

- Copy the head pointer into a temporary pointer temp.

temp = head

Traverse the list until we find the desired data value.

```
while(temp -> data != val)
    temp = temp -> next;
```

- Check if this is the last node of the list. If it is so then we can't perform deletion.

```
if(temp -> next == NULL)
{
    return;
}
```

- Check if the node which is to be deleted, is the last node of the list, if it so then we have to make the next pointer of this node point to null so that it can be the new last node of the list.

```
if(temp -> next -> next == NULL)
{
    temp -> next = NULL;
```

}

- Otherwise, make the pointer ptr point to the node which is to be deleted. Make the next of temp point to the next of ptr. Make the previous of next node of ptr point to temp. free the ptr.

```
ptr = temp -> next;  
temp -> next = ptr -> next;  
ptr -> next -> prev = temp;  
free(ptr);
```

Algorithm

- **Step 1:** IF HEAD = NULL

Write UNDERFLOW

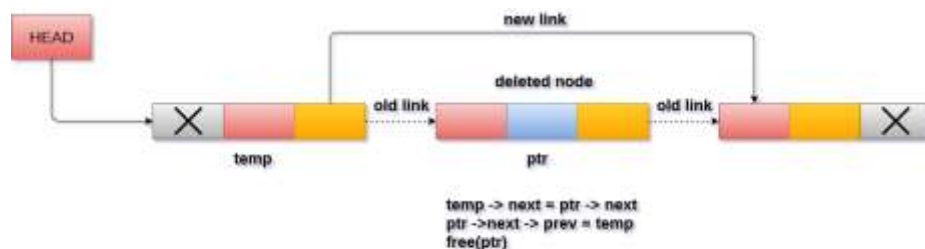
Go to Step 9

[END OF IF]

- **Step 2:** SET TEMP = HEAD
- **Step 3:** Repeat Step 4 while TEMP -> DATA != ITEM
- **Step 4:** SET TEMP = TEMP -> NEXT

[END OF LOOP]

- **Step 5:** SET PTR = TEMP -> NEXT
- **Step 6:** SET TEMP -> NEXT = PTR -> NEXT
- **Step 7:** SET PTR -> NEXT -> PREV = TEMP
- **Step 8:** FREE PTR
- **Step 9:** EXIT



Deletion of a specified node in doubly linked list

Searching for a specific node in Doubly Linked List

We just need to traverse the list in order to search for a specific element in the list. Perform the following operations in order to search for a specific operation.

- Copy head pointer into a temporary pointer variable ptr.

ptr = head

- declare a local variable I and assign it to 0.

i=0

- Traverse the list until the pointer ptr becomes null. Keep shifting pointer to its next and increasing i by +1.
- Compare each element of the list with the item which is to be searched.
- If the item matched with any node value then the location of that value I will be returned from the function else NULL is returned.

Algorithm

- **Step 1:** IF HEAD == NULL

WRITE "UNDERFLOW"

GOTO STEP 8

[END OF IF]

- **Step 2:** Set PTR = HEAD
- **Step 3:** Set i = 0
- **Step 4:** Repeat step 5 to 7 while PTR != NULL
- **Step 5:** IF PTR → data = item

return

i

[END OF IF]

- **Step 6:** i = i + 1
- **Step 7:** PTR = PTR → next
- **Step 8:** Exit

Traversing in doubly linked list

Traversing is the most common operation in case of each data structure. For this purpose, copy the head pointer in any of the temporary pointer ptr.

Ptr = head

then, traverse through the list by using while loop. Keep shifting value of pointer variable **ptr** until we find the last node. The last node contains **null** in its next part.

```
while(ptr != NULL)
{
    printf("%d\n",ptr->data);
    ptr=ptr->next;
}
```

Although, traversing means visiting each node of the list once to perform some specific operation. Here, we are printing the data associated with each node of the list.

Algorithm

- **Step 1:** IF HEAD == NULL

```
    WRITE "UNDERFLOW"
    GOTO STEP 6
[END OF IF]
```

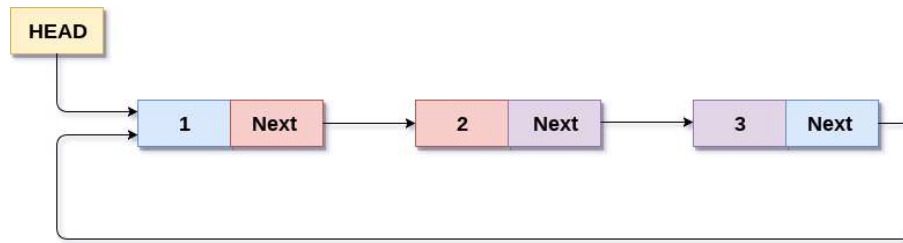
- **Step 2:** Set PTR = HEAD
- **Step 3:** Repeat step 4 and 5 while PTR != NULL
- **Step 4:** Write PTR → data
- **Step 5:** PTR = PTR → next
- **Step 6:** Exit

Circular Singly Linked List

In a circular Singly linked list, the last node of the list contains a pointer to the first node of the list. We can have circular singly linked list as well as circular doubly linked list.

We traverse a circular singly linked list until we reach the same node where we started. The circular singly linked list has no beginning and no ending. There is no null value present in the next part of any of the nodes.

The following image shows a circular singly linked list.



Circular Singly Linked List

Circular linked list are mostly used in task maintenance in operating systems. There are many examples where circular linked list are being used in computer science including browser surfing where a record of pages visited in the past by the user, is maintained in the form of circular linked lists and can be accessed again on clicking the previous button.

Operations on Circular Singly linked list:

Insertion

SN	Operation	Description
1	Insertion at beginning	Adding a node into circular singly linked list at the beginning.
2	Insertion at the end	Adding a node into circular singly linked list at the end.

Deletion & Traversing

SN	Operation	Description
1	Deletion at beginning	Removing the node from circular singly linked list at the beginning.
2	Deletion at the end	Removing the node from circular singly linked list at the end.
3	Searching	Compare each element of the node with the given item and return the location at which the item is present in the list otherwise return null.

4	<u>Traversing</u>	Visiting each element of the list at least once in order to perform some specific operation.
---	-------------------	--

Insertion into circular singly linked list at beginning

There are two scenarios in which a node can be inserted in a circular singly linked list at the beginning. Either the node will be inserted in an empty list or the node is to be inserted in an already filled list.

Firstly, allocate the memory space for the new node by using the malloc method of C language.

1. `struct node *ptr = (struct node *)malloc(sizeof(struct node));`

In the first scenario, the condition **head == NULL** will be true. Since, the list in which, we are inserting the node is a circular singly linked list, therefore the only node of the list (which is just inserted into the list) will point to itself only. We also need to make the head pointer point to this node. This will be done by using the following statements.

1. `if(head == NULL)`
2. `{`
3. `head = ptr;`
4. `ptr -> next = head;`
5. `}`

In the second scenario, the condition `head == NULL` will become false which means that the list contains at least one node. In this case, we need to traverse the list in order to reach the last node of the list. This will be done by using the following statement.

1. `temp = head;`
2. `while(temp->next != head)`
3. `temp = temp->next;`

At the end of the loop, the pointer temp would point to the last node of the list. Since, in a circular singly linked list, the last node of the list contains a pointer to the first node of the list. Therefore, we need to make the next pointer of the last node point to the head node of the list and the new node which is being inserted into the list will be the new head node of the list therefore the next pointer of temp will point to the new node ptr.

This will be done by using the following statements.

1. `temp -> next = ptr;`

the next pointer of temp will point to the existing head node of the list.

1. ptr->next = head;

Now, make the new node ptr, the new head node of the circular singly linked list.

1. head = ptr;

in this way, the node ptr has been inserted into the circular singly linked list at beginning.

Algorithm

- **Step 1:** IF PTR = NULL

Write OVERFLOW

Go to Step 11

[END OF IF]

- **Step 2:** SET NEW_NODE = PTR
- **Step 3:** SET PTR = PTR -> NEXT
- **Step 4:** SET NEW_NODE -> DATA = VAL
- **Step 5:** SET TEMP = HEAD
- **Step 6:** Repeat Step 8 while TEMP -> NEXT != HEAD
- **Step 7:** SET TEMP = TEMP -> NEXT

[END OF LOOP]

- **Step 8:** SET NEW_NODE -> NEXT = HEAD
- **Step 9:** SET TEMP → NEXT = NEW_NODE
- **Step 10:** SET HEAD = NEW_NODE
- **Step 11:** EXIT



There are two scenarios in which a node can be inserted in a circular singly linked list at the beginning. Either the node will be inserted in an empty list or the node is to be inserted in an already filled list.

```
1. struct node *ptr = (struct node *)malloc(sizeof(struct node));
```

```

1.  if(head == NULL)
2.      {
3.          head = ptr;
4.          ptr -> next = head;
5.      }

```

1. temp = head;
2. **while**(temp->next != head)
3. temp = temp->next;

At the end of the loop, the pointer temp would point to the last node of the list. Since, the new node which is being inserted into the list will be the new last node of the list. Therefore the

existing last node i.e. **temp** must point to the new node **ptr**. This is done by using the following statement.

1. `temp -> next = ptr;`

The new last node of the list i.e. `ptr` will point to the head node of the list.

1. `ptr -> next = head;`

In this way, a new node will be inserted in a circular singly linked list at the beginning.

Algorithm

- **Step 1:** IF PTR = NULL

Write

OVERFLOW

Go

to

Step

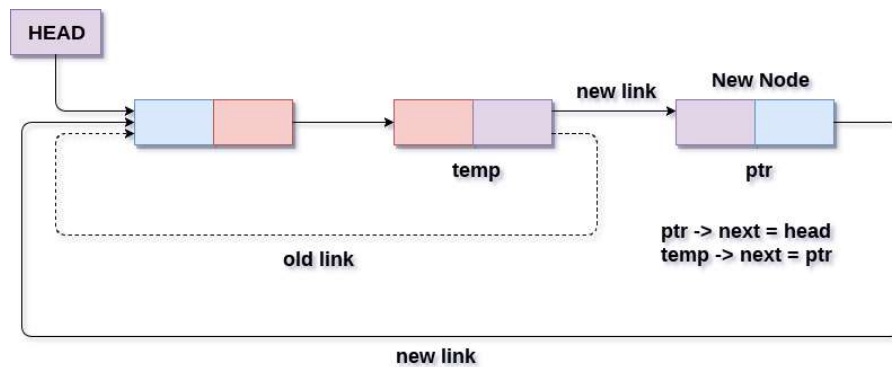
1

[END OF IF]

- **Step 2:** SET NEW_NODE = PTR
- **Step 3:** SET PTR = PTR -> NEXT
- **Step 4:** SET NEW_NODE -> DATA = VAL
- **Step 5:** SET NEW_NODE -> NEXT = HEAD
- **Step 6:** SET TEMP = HEAD
- **Step 7:** Repeat Step 8 while TEMP -> NEXT != HEAD
- **Step 8:** SET TEMP = TEMP -> NEXT

[END OF LOOP]

- **Step 9:** SET TEMP -> NEXT = NEW_NODE
- **Step 10:** EXIT



Insertion into circular singly linked list at end

Insertion into circular singly linked list at the end

There are two scenarios in which a node can be inserted in a circular singly linked list at the beginning. Either the node will be inserted in an empty list or the node is to be inserted in an already filled list.

□ Firstly, allocate the memory space for the new node by using the malloc method of C language.

1. `struct node *ptr = (struct node *)malloc(sizeof(struct node));`

In the first scenario, the condition **head == NULL** will be true. Since, the list in which, we are inserting the node is a circular singly linked list, therefore the only node of the list (which is just inserted into the list) will point to itself only. We also need to make the head pointer point to this node. This will be done by using the following statements.

1. `if(head == NULL)`
2. `{`
3. `head = ptr;`
4. `ptr -> next = head;`
5. `}`

In the second scenario, the condition **head == NULL** will become false which means that the list contains at least one node. In this case, we need to traverse the list in order to reach the last node of the list. This will be done by using the following statement.

1. `temp = head;`
2. `while(temp->next != head)`
3. `temp = temp->next;`

At the end of the loop, the pointer **temp** would point to the last node of the list. Since, the new node which is being inserted into the list will be the new last node of the list. Therefore the

existing last node i.e. **temp** must point to the new node **ptr**. This is done by using the following statement.

1. `temp -> next = ptr;`

The new last node of the list i.e. **ptr** will point to the head node of the list.

1. `ptr -> next = head;`

In this way, a new node will be inserted in a circular singly linked list at the beginning.

Algorithm

- **Step 1:** IF PTR = NULL

Write

Go

to

Step

OVERFLOW

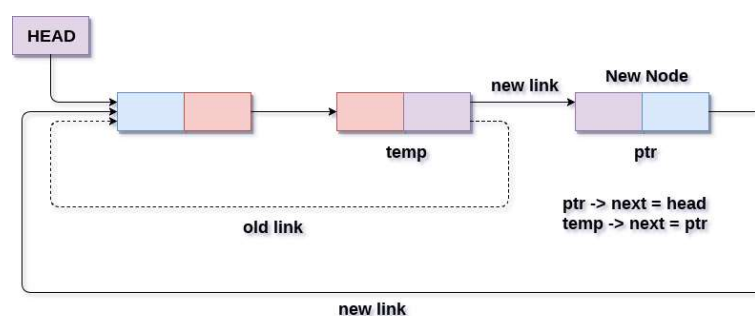
1

[END OF IF]

- **Step 2:** SET NEW_NODE = PTR
- **Step 3:** SET PTR = PTR -> NEXT
- **Step 4:** SET NEW_NODE -> DATA = VAL
- **Step 5:** SET NEW_NODE -> NEXT = HEAD
- **Step 6:** SET TEMP = HEAD
- **Step 7:** Repeat Step 8 while TEMP -> NEXT != HEAD
- **Step 8:** SET TEMP = TEMP -> NEXT

[END OF LOOP]

- **Step 9:** SET TEMP -> NEXT = NEW_NODE
- **Step 10:** EXIT



Insertion into circular singly linked list at end

Deletion in circular singly linked list at beginning

In order to delete a node in circular singly linked list, we need to make a few pointer adjustments.

There are three scenarios of deleting a node from circular singly linked list at beginning.

Scenario 1: (The list is Empty)

If the list is empty then the condition **head == NULL** will become true, in this case, we just need to print **underflow** on the screen and make exit.

```
1. if(head == NULL)
2. {
3.     printf("\nUNDERFLOW");
4.     return;
5. }
```

Scenario 2: (The list contains single node)

If the list contains single node then, the condition **head → next == head** will become true. In this case, we need to delete the entire list and make the head pointer free. This will be done by using the following statements.

```
1. if(head->next == head)
2. {
3.     head = NULL;
4.     free(head);
5. }
```

Scenario 3: (The list contains more than one node)

If the list contains more than one node then, in that case, we need to traverse the list by using the pointer **ptr** to reach the last node of the list. This will be done by using the following statements.

```
1. ptr = head;
2.     while(ptr -> next != head)
3.         ptr = ptr -> next;
```

At the end of the loop, the pointer **ptr** point to the last node of the list. Since, the last node of the list points to the head node of the list. Therefore this will be changed as now, the last node of the list will point to the next of the head node.

1. `ptr->next = head->next;`

Now, free the head pointer by using the `free()` method in C language.

1. `free(head);`

Make the node pointed by the next of the last node, the new head of the list.

1. `head = ptr->next;`

In this way, the node will be deleted from the circular singly linked list from the beginning.

Algorithm

- **Step 1:** IF HEAD = NULL

Write UNDERFLOW

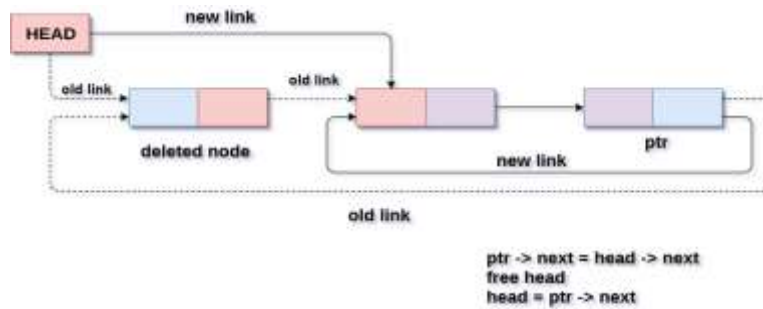
Go to Step 8

[END OF IF]

- **Step 2:** SET PTR = HEAD
- **Step 3:** Repeat Step 4 while $\text{PTR} \rightarrow \text{NEXT} \neq \text{HEAD}$
- **Step 4:** SET PTR = PTR $\rightarrow$ next

[END OF LOOP]

- **Step 5:** SET PTR $\rightarrow$ NEXT = HEAD $\rightarrow$ NEXT
- **Step 6:** FREE HEAD
- **Step 7:** SET HEAD = PTR $\rightarrow$ NEXT
- **Step 8:** EXIT



Deletion in circular singly linked list at beginning

Deletion in Circular singly linked list at the end

There are three scenarios of deleting a node in circular singly linked list at the end.

Scenario 1 (the list is empty)

If the list is empty then the condition **head == NULL** will become true, in this case, we just need to print **underflow** on the screen and make exit.

1. **if**(head == NULL)
2. {
3. printf("\nUNDERFLOW");
4. **return**;
5. }

Scenario 2(the list contains single element)

If the list contains single node then, the condition **head → next == head** will become true. In this case, we need to delete the entire list and make the head pointer free. This will be done by using the following statements.

1. **if**(head->next == head)
2. {
3. head = NULL;
4. free(head);
5. }

Scenario 3(the list contains more than one element)

If the list contains more than one element, then in order to delete the last element, we need to reach the last node. We also need to keep track of the second last node of the list. For this purpose, the two pointers ptr and preptr are defined. The following sequence of code is used for this purpose.

1. ptr = head;
2. **while**(ptr ->next != head)
3. {
4. preptr=ptr;
5. ptr = ptr->next;
6. }

now, we need to make just one more pointer adjustment. We need to make the next pointer of preptr point to the next of ptr (i.e. head) and then make pointer ptr free.

1. preptr->next = ptr -> next;
2. free(ptr);

Algorithm

- **Step 1:** IF HEAD = NULL

Write UNDERFLOW

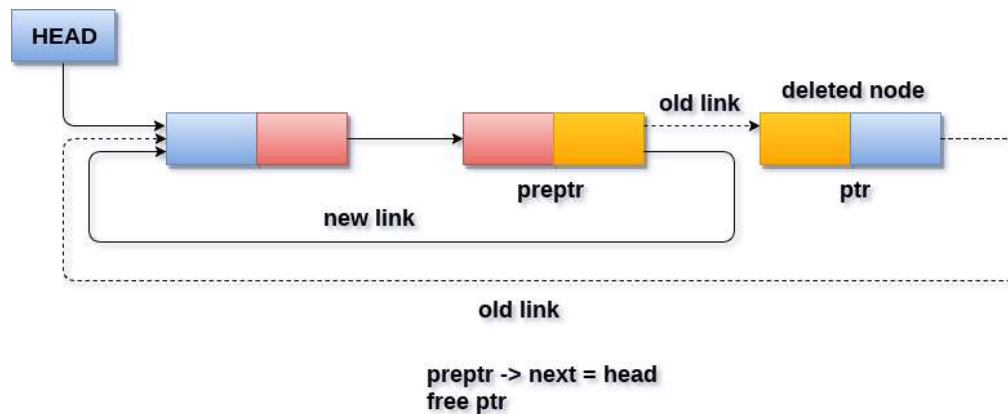
Go to Step 8

[END OF IF]

- **Step 2:** SET PTR = HEAD
- **Step 3:** Repeat Steps 4 and 5 while PTR -> NEXT != HEAD
- **Step 4:** SET PREPTR = PTR
- **Step 5:** SET PTR = PTR -> NEXT

[END OF LOOP]

- **Step 6:** SET PREPTR -> NEXT = HEAD
- **Step 7:** FREE PTR
- **Step 8:** EXIT



Deletion in circular singly linked list at end

Searching in circular singly linked list

Searching in circular singly linked list needs traversing across the list. The item which is to be searched in the list is matched with each node data of the list once and if the match found then the location of that item is returned otherwise -1 is returned.

The algorithm and its implementation in C is given as follows.

Algorithm

- **Step 1:** SET PTR = HEAD
- **Step 2:** Set I = 0
- **STEP 3:** IF PTR = NULL

WRITE "EMPTY LIST"

GOTO STEP 8

END OF IF

- **STEP 4:** IF HEAD → DATA = ITEM

WRITE i+1 RETURN [END OF IF]

- **STEP 5:** REPEAT STEP 5 TO 7 UNTIL PTR->next != head
- **STEP 6:** if ptr → data = item

write i+1

RETURN

End of IF

- **STEP 7:** $I = I + 1$
- **STEP 8:** $PTR = PTR \rightarrow NEXT$

[END OF LOOP]

- **STEP 9:** EXIT

Traversing in Circular Singly linked list

Traversing in circular singly linked list can be done through a loop. Initialize the temporary pointer variable **temp** to head pointer and run the while loop until the next pointer of temp becomes **head**. The algorithm and the c function implementing the algorithm is described as follows.

Algorithm

- **STEP 1:** SET $PTR = HEAD$
- **STEP 2:** IF $PTR = NULL$

WRITE "EMPTY LIST"

GOTO STEP 8

END OF IF

- **STEP 4:** REPEAT STEP 5 AND 6 UNTIL $PTR \rightarrow NEXT \neq HEAD$
- **STEP 5:** PRINT $PTR \rightarrow DATA$
- **STEP 6:** $PTR = PTR \rightarrow NEXT$

[END OF LOOP]

- **STEP 7:** PRINT $PTR \rightarrow DATA$
- **STEP 8:** EXIT

Application of Linked List

In this article, we will understand the linked list applications in detail.

What do you mean by Linked list?

A linked list is a linear data structure consisting of elements called nodes where each node is composed of two parts: an information part and a link part, also called the next pointer part.

Linked list is used in a wide variety of applications such as

- Polynomial Manipulation representation
- Addition of long positive integers
- Representation of sparse matrices
- Addition of long positive integers
- Symbol table creation
- Mailing list
- Memory management
- Linked allocation of files
- Multiple precision arithmetic etc

Polynomial Manipulation

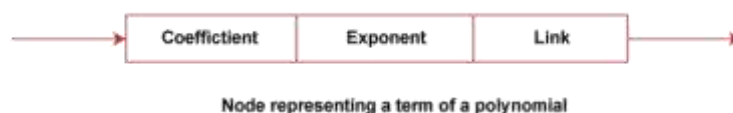
Polynomial manipulations are one of the most important applications of linked lists. Polynomials are an important part of mathematics not inherently supported as a data type by most languages. A polynomial is a collection of different terms, each comprising coefficients, and exponents. It can be represented using a linked list. This representation makes polynomial manipulation efficient.

While representing a polynomial using a linked list, each polynomial term represents a node in the linked list. To get better efficiency in processing, we assume that the term of every polynomial is stored within the linked list in the order of decreasing exponents. Also, no two terms have the same exponent, and no term has a zero coefficient and without coefficients. The coefficient takes a value of 1.

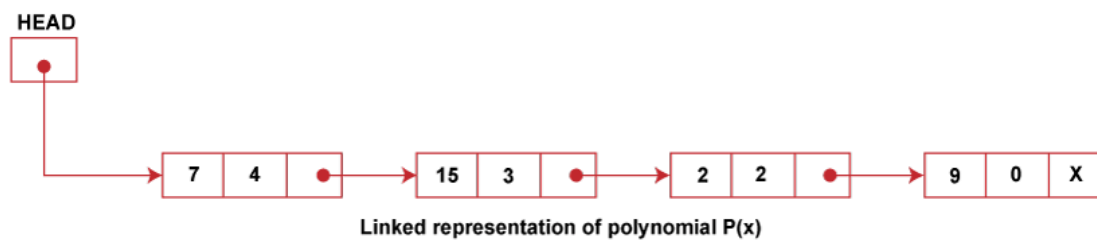
Each node of a linked list representing polynomial constitute three parts:

- The first part contains the value of the coefficient of the term.
- The second part contains the value of the exponent.
- The third part, LINK points to the next term (next node).

The structure of a node of a linked list that represents a polynomial is shown below:



Consider a polynomial $P(x) = 7x^4 + 15x^3 - 2x^2 + 9$. Here 7, 15, -2, and 9 are the coefficients, and 4,3,2,0 are the exponents of the terms in the polynomial. On representing this polynomial using a linked list, we have



Observe that the number of nodes equals the number of terms in the polynomial. So we have 4 nodes. Moreover, the terms are stored to decrease exponents in the linked list. Such representation of polynomial using linked lists makes the operations like subtraction, addition, multiplication, etc., on polynomial very easy.

Addition of Polynomials:

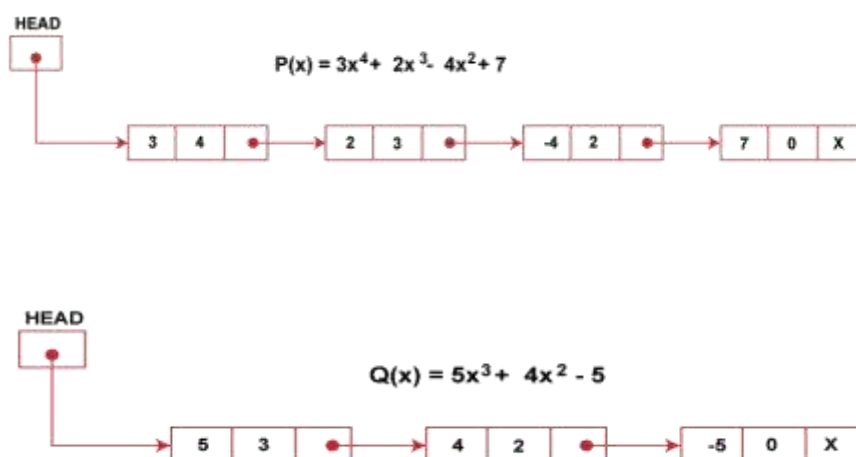
To add two polynomials, we traverse the list P and Q. We take corresponding terms of the list P and Q and compare their exponents. If the two exponents are equal, the coefficients are added to create a new coefficient. If the new coefficient is equal to 0, then the term is dropped, and if it is not zero, it is inserted at the end of the new linked list containing the resulting polynomial. If one of the exponents is larger than the other, the corresponding term is immediately placed into the new linked list, and the term with the smaller exponent is held to be compared with the next term from the other list. If one list ends before the other, the rest of the terms of the longer list is inserted at the end of the new linked list containing the resulting polynomial.

Let us consider an example an example to show how the addition of two polynomials is performed,

$$P(x) = 3x^4 + 2x^3 - 4x^2 + 7$$

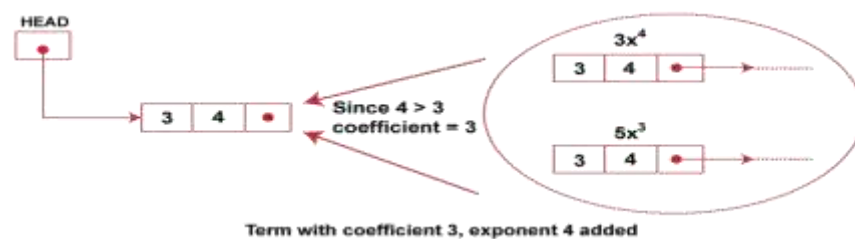
$$Q(x) = 5x^3 + 4x^2 - 5$$

These polynomials are represented using a linked list in order of decreasing exponents as follows:

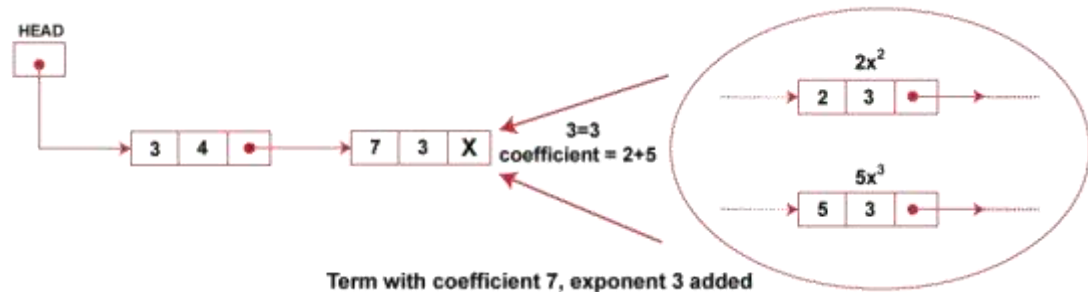


To generate a new linked list for the resulting polynomials that is formed on the addition of given polynomials $P(x)$ and $Q(x)$, we perform the following steps,

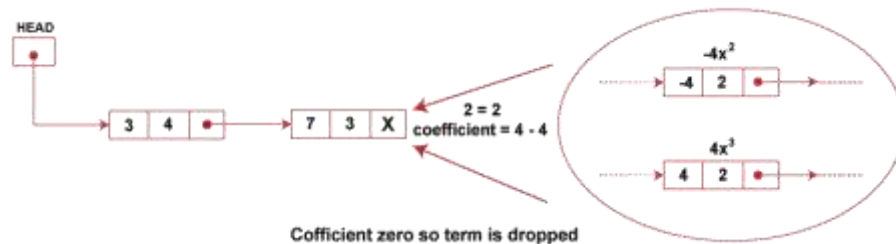
1. Traverse the two lists P and Q and examine all the nodes.
2. We compare the exponents of the corresponding terms of two polynomials. The first term of polynomials P and Q contain exponents 4 and 3, respectively. Since the exponent of the first term of the polynomial P is greater than the other polynomial Q, the term having a larger exponent is inserted into the new list. The new list initially looks as shown below:



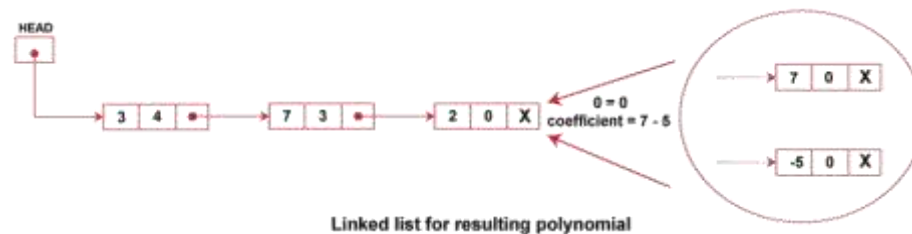
3. We then compare the exponent of the next term of the list P with the exponents of the present term of list Q. Since the two exponents are equal, so their coefficients are added and appended to the new list as follows:



4. Then we move to the next term of P and Q lists and compare their exponents. Since exponents of both these terms are equal and after addition of their coefficients, we get 0, so the term is dropped, and no node is appended to the new list after this,



5. Moving to the next term of the two lists, P and Q, we find that the corresponding terms have the same exponents equal to 0. We add their coefficients and append them to the new list for the resulting polynomial as shown below:



Some other applications of linked list:

- **Memory Management:** Memory management is one of the operating system's key features. It decides how to allocate and reclaim storage for processes running on the system. We can use a linked list to keep track of portions of memory available for allocation.
- **Mailing List:** Linked lists have their use in email applications. Since it is difficult to predict multiple lists, maybe a mailer builds a linked list of addresses before sending a message.
- **LISP:** LISP is an acronym for LIST processor, an important programming language in artificial intelligence. This language extensively uses linked lists in performing symbolic processing.
- **Linked allocation of files:** A file of large size may not be stored in one place on a disk. So there must be some mechanism to link all the scattered parts of the file together. The use of a linked list allows an efficient file allocation method in which each block of a file contains a pointer to the file's text block. But this method is good only for sequential access.
- **Virtual Memory:** An interesting application of linked lists is found in the way systems support virtual memory.
- **Support for other data structures:** Some other data structures like stacks, queues, hash tables, graphs can be implemented using a linked list.